

开发/测试环境

本次实验是在原生 linux 环境进行，操作系统发行版为 Cachyos，qemu 版本为 9.2.3。

Figure 1: 本地开发环境

实验细节

ch3 中只有一个任务，就是实现一个 `trace` 系统调用，来跟踪系统调用的执行情况。按照调用规范所给的文档，我把这个大的问题进行了拆分，分成了几个小问题来解决。

```
1 match trace_request {  
2     0 => todo!(), // 读取 id 地址处一个字节的无符号整数值  
3     1 => todo!(), // 写入 data(u8) 到该用户程序 id 地址处  
4     2 => todo!(), // 查询当前任务调用编号为 id 的系统调用的次数  
5     _ => -1,  
6 }
```

一、`trace_request=0`

考虑到目前，我们的操作系统中还没有虚拟内存的概念，所有的进程和操作系统都在同一块物理内存中，所以我们可以直接使用 `unsafe` 来进行内存的读写操作。

二、`trace_request=1`

同样的，我们可以直接使用 `unsafe` 来进行内存的读写操作。

三、`trace_request=2`

这是本次任务的重点，主要是对系统调用的次数进行统计。简单分析一下问题：

1. 需要一个数据结构来存储系统调用的次数。
2. 记录的这个状态是进程级别的，所以需要在进程的上下文中进行存储。

3. 需要使用系统调用号进行索引。

通过上面三点分析，可以确定我们需要在 `os/src/task/task.rs` 下的 `TaskControlBlock` 结构体中维护一个 `syscalls_counter` 来记录系统调用的次数。接下来选定数据结构，但是我发现由于在操作系统中，我们并没有使用 `std` 库，所以我们不能使用 `HashMap` 来存储系统调用的次数。我们可以使用一个长度为 512 的数组来存储系统调用的次数，索引为系统调用号，值为系统调用的次数。虽然有些浪费，但是这样也可以实现对系统调用次数的统计了。所以确定类型为 `[u32; 512]`。

接下来的问题是，如何通过友好的 API 来访问这个数组。我定义了一个 `syscalls_cnt` 的 `getter` 和 `setter` 方法来访问这个数组。这样就可以通过系统调用号来访问这个数组了。不过按照已给的代码风格，我应该要写三层抽象：

1. 第一层抽象在 `TaskControlBlock` 上，会直接访问 `syscalls_cnt` 数组。
2. 第二层抽象在 `TASK_MANAGER` 上，提供一个处理当前任务的 `getter` 和 `setter` 方法。
3. 第三层抽象在 `task` 模块上，提供模块对外的访问接口。

最后在 `syscall` 模块中，当调用系统调用时，就调用 `task` 模块下的 `syscall_trace` 方法来记录系统调用的次数。这样就完成了对系统调用次数的统计。

当正式调用 `sys_trace` 时，调用 `task` 模块下的 `get_syscall_trace` 方法来获取系统调用次数。

问答题

问题一：

使用 `RustSBI version 0.4.0-alpha.1, adapting to RISC-V SBI v2.0.0`，可以看到，分别运行三个 bad 测例得到的结果：

```
1 # ch2b_bad_address.rs
2 [kernel] PageFault in application, bad addr = 0x0, bad instruction = 0x804003a4, kernel killed it.
3 # ch2b_bad_instructions.rs
4 [kernel] IllegalInstruction in application, kernel killed it.
5 # ch2b_bad_register.rs
6 [kernel] IllegalInstruction in application, kernel killed it.
```

第一个访问了一个不存在的地址，出现了页错误。第二个执行了 `sret` 指令，这不应该在用户态执行。第三个都是访问了一个 `sstatus`，也不应在用户态。

问题二：

1. 刚进入 `__restore` 时，`sp` 指向用户进程的栈顶，通过栈可以依次恢复之前上下文切换时保存的寄存器状态。`__restore` 会在用户态进行系统调用返回时被调用，恢复用户进程的上下文。也会在内核态进行 `sbi_call` 时被调用。

2. 特殊处理了三个寄存器：

- `sstatus`：存储处理器的状态信息，包括当前的特权级别、中断使能状态等，这里的话，其中的 `SPP` 字段会被修改为 CPU 当前的特权级（U/S）。
- `sepc`：存储异常返回时的程序计数器（PC）值，即异常发生时的指令地址，这样就可以在异常返回时继续保持执行流
- `sscratch`：存储内核态和用户态切换时的上下文信息，通常用于保存内核栈指针或其他临时数据，比如这里，它保存了用户栈指针。

3. `x2` 是栈指针(Stack pointer)，在 `__restore` 最后的 `csrrw sp, sscratch, sp` 才会恢复到用户栈指针。`x4` 是线程指针(Thread pointer)，目前我们的操作系统中还没有这个概念。

4. 根据《The RISC-V reader》`csrrw sp, sscratch, sp` 执行的语义如下：

```
1 t = sscratch;
2 sscratch = sp;
3 sp = t;
```

也就是说，它们相互交换了值。原来 `sp` 中存储的是内核栈指针，而 `sscratch` 中存储的是用户栈指针，所以交换后，`sp` 中存储的是用户栈指针，`sscratch` 中存储的是内核栈指针。

5. 状态切换发生在 `sret` 指令。因为它会进行如下处理

1. `sstatus.SPP` 置为 0。
2. `sstatus.SIE` 设置为 `sstatus.SPIE`
3. `sstatus.SPIE` 设置为 1。
4. 最后 `pc` 设置为 `sepc`，也就是异常返回时的程序计数器（PC）值。

6. 这里运行完成后的 `sp`，`sscratch` 与 4 中恰好相反。

7. 调用 `ecall` 指令时就从 U 态进入了 S 态。